

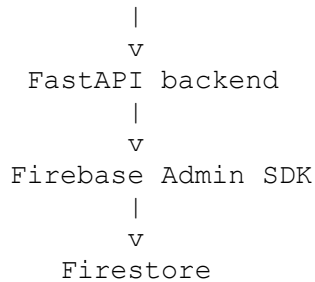
PhysioVision AI Database Schema

Database technology

PhysioVision AI uses Cloud Firestore, Firebase's NoSQL document database. The mobile application sends session data to the FastAPI backend, and the backend uses the Firebase Admin SDK to write and read Firestore documents.

```text

React Native / Expo mobile app



### Collections and documents

`users/{user\_id}`

Each authenticated user has one profile document. The document ID is the Firebase user ID (`uid`).

| Field        | Type           | Description                                      |
|--------------|----------------|--------------------------------------------------|
| `user_id`    | string         | Unique Firebase or demonstration user identifier |
| `email`      | string or null | User email address                               |
| `name`       | string or null | Display name                                     |
| `updated_at` | timestamp      | Server time when the profile was updated         |

`users/{user\_id}/exercise\_sessions/{session\_id}`

Each completed exercise creates a new document in the user's `exercise\_sessions` subcollection. Firestore generates `session\_id` when the document is created.

| Field              | Type           | Description                                  |
|--------------------|----------------|----------------------------------------------|
| `session_id`       | string         | Unique Firestore document identifier         |
| `user_id`          | string         | Owner of the exercise session                |
| `user_email`       | string or null | Email copied for history display             |
| `user_name`        | string or null | Name copied for history display              |
| `exercise_id`      | string         | Stable application exercise identifier       |
| `exercise_name`    | string         | Normalised exercise name used by the backend |
| `reps`             | integer        | Number of completed repetitions              |
| `duration_seconds` | integer        | Session duration in seconds                  |
| `accuracy_score`   | number         | Form score from the analysis session         |
| `created_at`       | timestamp      | Firestore server creation time               |

### Example document path

```
text
users/demo-user-polycarp/
 exercise_sessions/abc123
...
```

## Example session document

```
```json
{
  "session_id": "abc123",
  "user_id": "demo-user-polycarp",
  "user_email": "polycarp@example.com",
  "user_name": "Polycarp",
  "exercise_id": "squats",
  "exercise_name": "squat",
  "reps": 12,
  "duration_seconds": 46,
  "accuracy_score": 95,
  "created_at": "Firestore server timestamp"
}
```

Data operations

Operation	Application path	Firestore operation
Create/update profile	`POST /auth/register` or `POST /auth/login`	Set `users/{user_id}` with merge
Save session	`POST /api/save-session`	Create a document in `exercise_sessions`
Read history	`GET /api/sessions/{user_id}`	Read sessions ordered by `created_at` descending

Organisation and normalization

Firestore is document-oriented, so relational normal forms are not applied directly. The design separates each user into a unique parent document and stores that user's sessions in a subcollection. This prevents unrelated users' sessions from being mixed and makes user history retrieval straightforward.

The session document includes `user\_email` and `user\_name` as controlled denormalization. These fields are duplicated to display history without an additional profile lookup. If profile editing is added in the future, the application should update these copied values or use a separate display lookup.

Security and deployment notes

- The Firebase service-account key is backend-only and must never be placed in the mobile application or included in the submission archive.

- Production deployments must verify Firebase bearer tokens on session routes and derive the user ID from the verified token.
- Demo authentication is for presentation only and must be disabled for production use.
- Firestore security rules, restricted CORS, HTTPS, and hosting-provider environment variables are required for public deployment.